Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Examples: Using the API to Retrieve Case Details

You can use the Stellar Cyber API to query the DP for a detailed list of Cases, optionally filtered by tenant, assignee, tags, priority, status, and score. You can also specify that the Cases returned by the API be sorted according to a specified parameter.

Refer to **Configuring API Authentication** for general requirements to use the API.

API Syntax for Retrieving Case Details

The syntax for retrieving case details via the API is as follows:

```
https://URL/connect/api/v1/cases
```

Querying for a Specific Case's Details

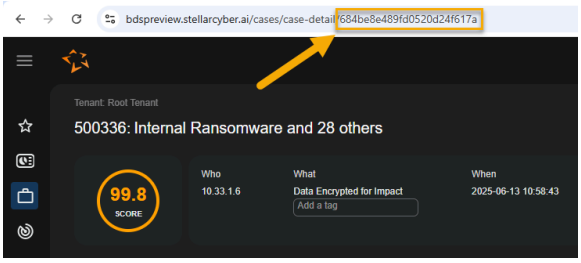
You can also query the API for a specific case's details by providing its internal Case ID:

```
https://URL/connect/api/v1/cases/{case_id}
```

Finding the Case ID for the Query

To query for a specific case's details, you must first have the case's ID. You can retrieve an case's ID using either of the following techniques:

- Navigate to the Detail display for the case whose ID you want to retrieve. The case's ID is included in the URL for the Detail page, as shown below.



- You can also retrieve the case ID through the cases API. Refer to the examples and instructions in **Examples: Using the API to Retrieve Case Details** for details on how

to fetch cases through the API.

Once you've retrieved a case whose details you want to fetch, locate the `"_id": "{case_id}"` field. This is the ID you must supply when querying for a specific case's details.

Arguments:

You can optionally limit the API call using any combination of the arguments in the API reference. Note the following:

- All arguments are optional.
- Arguments are case-sensitive.
- If you specify multiple arguments, they must be separated by a **&** in the command line with no intervening spaces.
- Arguments of different types are joined with a logical **AND** – only records matching all supplied arguments are returned by the API call.
- Multiple values specified for a single argument type are joined with a logical OR. For example, if you supply multiple **tags**, records matching any of the supplied tags are returned. Note that **tags** is the only parameter for the Cases API that can take multiple values, separated by commas.
- The maximum number of cases returned by the cases API is 500. You can use the limit command described in the table below to return even fewer cases. You cannot, however, set the limit command to a value greater than 500.

Types of API Calls for Case Details

Depending on how you formulate the API call using the arguments listed above, you can make the following types of queries. Examples of each are provided following the summary below. Values in bold are the values you must supply as part of the query.

- Query for the first 250 cases in the DP's list:

  `https://<Stellar Cyber_IP>/connect/api/v1/cases?limit=`**`250`**

- Query for the 10 cases associated with a specific tenant with the highest case score:

  `https://URL/connect/api/v1/cases?cust_id=<`**`tenant_id`**`>?sort=`**`case_score`**`&order=`**`desc`**`&limit=`**`10`**

- Query for all cases associated with a specific tenant above a specific score:

  `https://URL/connect/api/v1/cases?tenant_id=<`**`tenant_id`**`>&min_score=<`**`min_score`**`>`

In response, the API returns detailed information on all cases matching the API call, sorted as requested, if you include **sort** and **order** parameters. The information returned is similar to that presented in the Cases page's tabular view and is explained below in **Case Information Returned by the API**.

Finding Parameter Values for Your Case Query

Several of the examples below require you to use case parameters in your query. You can find most case parameters in the Cases page displayed in Table mode:

1. Open the **Cases** page.
2. Locate the table entry for the case for which you want to query.
3. If the parameter you want to use as a filter is not already shown as a column header, you can add it now by clicking the Columns entry at the left of the table and selecting the field you want to add as a column.
4. Right-click in the cell you want to use as a filter and use the **Copy to Clipboard** command in the context menu that appears to copy the parameter.
5. Paste the parameter value into a text file to store it temporarily.

Examples

The following sections provide examples of common ways to use the cases API. The examples all use Python, though any programming language will work.

Querying for Cases Across All Tenants

The following example uses a Python script to query for the first 250 cases across all tenants with the following details:

- **Stellar Cyber DP IP Address** – myserver.stellarcyber.cloud
- **API Key (Refresh Token) from GUI** – refresh_token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZXJfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYjQyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2SenlwUTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo
- **limit** – 250

Understanding the Script

This script works as follows:

- The script sets the **host** and **refresh_token** parameters in **Step 1** in the sample.
- Because JWTs expire ten minutes after they are generated, this script includes logic that generates and uses a fresh JWT every time the script is run. The script runs the **getAccessToken** procedure to generate the new JWT (**Step 2** in the sample).
- The script uses the generated JWT to make a call to the cases API in the **getCases** procedure (**Step 3** in the sample). The limit parameter is included in the URL and limits the number of cases returned to 250.
- The script also prints the generated JWT to the screen. This, however, is not strictly necessary since the **getAccessToken** procedure already prints the status code for the call to the **access_token** API (**200** for success; **401** for failure).

```
#!/usr/bin/python3
import requests
```

```
import base64
import json
from urllib.parse import urlunparse
requests.packages.urllib3.disable_warnings()

# Step 1
# Add DP IP/hostname and refresh token from GUI here
HOST = "myserver.stellarcyber.cloud"
userid = "myuser@stellarcyber.ai"
refresh_token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZX
JfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYj
QyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2Senlw
UTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo"

def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]


def getCases(token):
    headers = {"Authorization": "Bearer " + token}
    url = urlunparse(("https", HOST, "/connect/api/v1/cases?limit=250", "", "", ""))
    res = requests.get(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()

if __name__ == "__main__":

    # Step 2: Use getAccessToken with supplied credentials to generate JWT
    jwt = getAccessToken(refresh_token)
    print("------------ jwt ------------")
    print(jwt)
    print("------------ jwt  end ------------")

    # Step 3: use JWT token to call public API
    cases = getCases(jwt)
    print("------------ call result of /connect/api/v1/cases ------------")
    print(cases)
    print("------------ end api results ------------")
```

Querying for Cases Above a Minimum Score

The following example uses a Python script to query for 25 cases above a minimum Case Score of 90, sorted in descending order. The example uses the following details:

- **Stellar Cyber DP IP Address** – myserver.stellarcyber.cloud
- **API Key (Refresh Token) from GUI** – eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZXJfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYjQyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2SenlwUTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo
- min_score – 90

Understanding the Script

This script works as follows:

- The script sets the **host** and **refresh_token** parameters in **Step 1** in the sample.
- Because JWTs expire ten minutes after they are generated, this script includes logic that generates and uses a fresh JWT every time the script is run. The script runs the **getAccessToken** procedure to generate the new JWT (**Step 2** in the sample).

- The script uses the generated JWT to make a call to the cases API in the **getCases** procedure (**Step 3** in the sample). The URL includes the **min_score** of 90, the **limit** of 25, and the sort criteria and order.
- The script also prints the generated JWT to the screen. This, however, is not strictly necessary since the **getAccessToken** procedure already prints the status code for the call to the **access_token** API (**200** for success; **401** for failure).

```
#!/usr/bin/python3

import requests
import base64
import json
from urllib.parse import urlunparse
requests.packages.urllib3.disable_warnings()

# Step 1
# Add DP IP/hostname and refresh token from GUI here
HOST = "myserver.stellarcyber.cloud"
refresh_token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZX
JfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYj
QyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2Senlw
UTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo"

def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]

def getCases(token):
    headers = {"Authorization": "Bearer " + token}
    url = urlunparse(("https", HOST, "/connect/api/v1/cases?
min_score=90&limit=25&sort=case_score&order=desc", "", "", ""))
    res = requests.get(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()

if __name__ == "__main__":

    # Step 2: Use getAccessToken with supplied credentials to generate JWT
    jwt = getAccessToken(refresh_token)
    print("------------ jwt -------------")
    print(jwt)
    print("------------ jwt  end -------------")

    # Step 3: use JWT token to call public API
    cases = getCases(jwt)
    print("------------ call result of /connect/api/v1/cases -------------")
    print(cases)
    print("------------ end api results -------------")
```

Querying for a Single Tenant's Cases

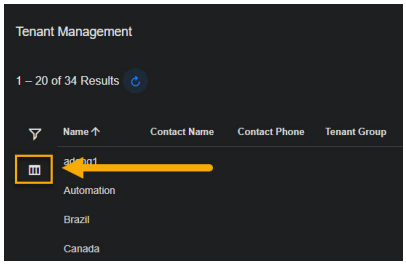
The following example uses a Python script to query for 5 cases belonging to a specific tenant with the following details:

- **Stellar Cyber DP IP Address** – myserver.stellarcyber.cloud
- **API Key (Refresh Token) from GUI** – eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZXJfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYjQyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2SenlwUTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo
- **Tenant ID** – c7cc6762d8dd4dafa7384f4e0240fc8d

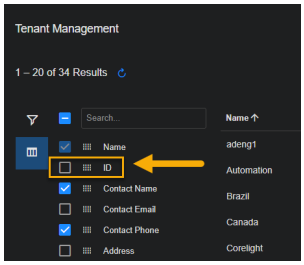
You must use the numerical ID for the tenant and not the display name from the user interface. You can retrieve this ID from the **System | ORGANIZATION MANAGEMENT | Tenants** page by adding the **ID** column to the display and copying the ID from the table.

Click here to see a procedure demonstrating how to do this:

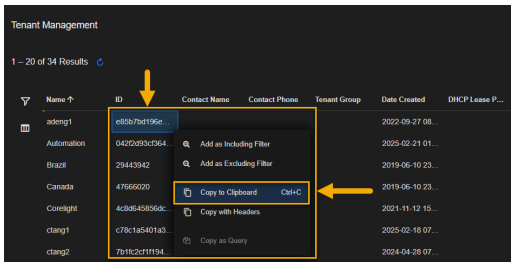
1. Navigate to the **System | ORGANIZATION MANAGEMENT | Tenants** page.
2. Click the **Columns** entry at the left of the table.



3. Check the **ID** box to add the numerical tenant ID to the table.



4. Locate the entry for the tenant whose ID you want to copy. We are copying the ID for the tenant with the name **Rui**.
5. Right-click the entry in the ID column for the tenant whose ID you want to copy and choose **Copy to Clipboard** from the context menu that appears. For example:



6. Paste the ID into a text file so you can have it handy for use in your scripts.

Understanding the Script

This script works as follows:

- The script sets the **host** and **refresh_token** parameters in **Step 1** in the sample.
- Because JWTs expire ten minutes after they are generated, this script includes logic that generates and uses a fresh JWT every time the script is run. The script runs the **getAccessToken** procedure to generate the new JWT (**Step 2** in the sample).
- The script uses the generated JWT to make a call to the cases API in the **getCases** procedure (**Step 3** in the sample). The Tenant ID and limit are both included in the URL.

- The script also prints the generated JWT to the screen. This, however, is not strictly necessary since the **getAccessToken** procedure already prints the status code for the call to the **access_token** API (**200** for success; **401** for failure).

```
#!/usr/bin/python3

import requests
import base64
import json
from urllib.parse import urlunparse
requests.packages.urllib3.disable_warnings()

# Step 1
# Add DP IP/hostname and refresh token from GUI here
HOST = "myserver.stellarcyber.cloud"
refresh_token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZX
JfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYj
QyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2Senlw
UTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo"

def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]


def getCases(token):
    headers = {"Authorization": "Bearer " + token}
    url = urlunparse(("https", HOST, "/connect/api/v1/cases?
tenant_id=c7cc6762d8dd4dafa7384f4e0240fc8d&limit=5", "", "", ""))
    res = requests.get(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()

if __name__ == "__main__":

    # Step 2: Use getAccessToken with supplied credentials to generate JWT
    jwt = getAccessToken(refresh_token)
    print("------------ jwt ----------")
    print(jwt)
    print("------------ jwt  end ----------")

    # Step 3: use JWT token to call public API
    cases = getCases(jwt)
    print("------------ call result of /connect/api/v1/cases ----------")
    print(cases)
    print("------------ end api results ----------")
```

Case Information Returned by the API

The API returns the following information for each case matching the API call in field:value pairs, with fields separated from values by a colon. Separate field:value pairs are separated with commas for easy import. Refer to **Sample Output** for an example.

API Field Name	Description
_id	The internal ID for the case. This is the ID you must use when fetching the case's observables via the API.
acknowledged	The time at which the case was first acknowledged, expressed in epoch time. A case is considered acknowledged when its status changes from **New** to anything else.
assignee	The resource assigned to this case in the Case Details page's Case Identification panel. All cases start out as **Unassigned**, but can be assigned to any user currently defined in the Stellar Cyber system.

API Field Name	Description
closed	Indicates whether the is closed (**1**) or not (**0**). A case is closed when its status is changed to either **Resolved** or **Cancelled**.
created_at	The time at which the case was created, expressed in epoch time.
created_by	The user account that created the case. Cases generated by Stellar Cyber show a value of **System** for this field.
cust_id	The ID of the Tenant to which the case belongs, if any.
modified_at	The last time the case was modified, expressed in epoch time.
modified_by	The internal ID of the account that performed the last modification of the case.
name	The friendly name of the case as it appears in Stellar Cyber displays.
score	The score for the case at the time it was retrieved via the API.
size	The number of alerts associated with the case.
status	The status associated with this case. Can be **New**, **Escalated**, **In Progress**, **Resolved**, or **Cancelled**. You can change a case's status in the Case Detail view.
severity	The severity of the case. Cases start out with a **Medium** severity, but can be changed in the Case Detail view to **Low**, **High**, or **Critical**.
tags	The tags assigned to this case in the Case Detail page's Case Identification Panel, if any.
ticket_id	The system-generated Ticket ID assigned to this case.
version	The API version used to return data for this query.
assignee_name	The account to which the case is assigned.
created_by_name	Indicates who created the case. Can be either **System** or a specific Stellar Cyber user account.
modified_by_name	The username of the last Stellar Cyber account that made a modification to the case.
tenant_name	The tenant with which the case is associated.
total	The total number of cases matching the query before limited by the **limit** and **skip** arguments.

Sample Output

The text below shows sample output of an API call that returned a single case:


```
{'data': {'total': 171298, 'cases': [{'_id': '655459dba3149258cd73014b',
'acknowledged': 1700102364585, 'assignee': 'e7fd8d407209477091862d3052bdcef8',
'closed': 0, 'created_at': 1700026843539, 'created_by': 'System', 'cust_id':
'c7cc6762d8dd4dafa7384f4e0240fc8d', 'modified_at': 1702287623433,
'modified_by': '32c8755be7014557a84b452e9ee981a0', 'name': 'Cynet: Data
Encrypted for Impact and 133 others', 'score': 100, 'size': 134, 'status': 'In
Progress', 'severity': 'Critical', 'tags': ['11'], 'ticket_id': 1667,
'version': 138, 'created_by_name': 'System', 'modified_by_name':
'admin1@custom1.com', 'assignee_name': '0-root_user@123.com', 'tenant_name':
'Rui'}]}}
```



© 2026 Stellar Cyber . All rights reserved.